MEDIFLOW · HACKATHON ONE G10 · BRASIL

Alinhamento de contrato

entre os dois repositórios do MediFlow

Leitura do repositório g10-brasil-equipe-15_mediflow (branch develop, 24 arquivos) contra os documentos confirmados do projeto: Especificação Funcional v1.0 atualizada, Análise de Requisitos v1.1 e o contrato de dados executável.

PARA	DE	DATA
Izabela — G10 Brasil · Equipe 15	Time de documentação e testes	22/10/2026

RESUMO EM TRÊS NÚMEROS

10 regras de negócio já estão corretas no seu grafo — nenhuma precisa ser reescrita.
7 ajustes de contrato (nomes de enums, campos e blocos que a resposta não expõe).
5 itens que ainda faltam implementar — dois deles são a sua parte dos buckets.

COMO LER ESTE DOCUMENTO

Cada ajuste traz a referência no documento que originou a exigência (§5.2, RN-03, RF-21…). Se a referência não bastar, o arquivo está nos links da última página.
Nada aqui é opinião: tudo é consequência direta dos documentos que o time já aprovou.
A ordem é a ordem sugerida de execução — as três primeiras destravam o resto.

1. O que já está correto

Seu grafo acertou todas as decisões que tomamos em 21/10 — inclusive uma em que a especificação original divergia do brief.

Antes de qualquer coisa: o print que circulou no grupo mostrava só a cauda dos testes. Vendo a lista completa — 25 funções de teste, sem contar as parametrizadas — a implementação está bem mais avançada do que parecia, e em três pontos, melhor que a nossa.

Regra	Referência	Onde está no seu código
Urgência isolada NÃO gera revisão	brief (R1)	test_urgencia_isolada_nao_exige_revisao
Conflito ALTA e MÉDIA revisam; BAIXA só registra	decisão A1	test_penalidade_por_conflito (0,70 / 0,80 / 0,90)
Fórmula do score de extração	decisão A3	idêntica: 1 − 0,2 − 0,1 − 0,3/0,2/0,1
Prioridade não identificada gera revisão	decisão A4	test_prioridade_ambigua_exige_revisao
Urgência INDETERMINADO conta como incerto	decisão A5	test_urgencia_indeterminada_exige_revisao
Score combinado = min(classificação, extração)	RF-18	avalia_qualidade()
5 campos obrigatórios; CID isolado NÃO revisa	RF-21	test_campo_obrigatorio_ausente + test_cid_ausente_isoladamente_nao_exige_revisao
Urgência + ambiguidade = dois destinos	§8.2	test_urgencia_com_ambiguidade_tem_dois_destinos
Justificativa e regra registradas em toda decisão	RN-05	test_decisao_tem_justificativa_e_regra
Grafo não confirma persistência	RF-30	etapa: DECISAO_CALCULADA

O PONTO MAIS DIFÍCIL, E VOCÊ ACERTOU

A §8.2 da Especificação Funcional v1.0 original manda revisar obrigatoriamente todo caso urgente. O brief do hackathon mostra o contrário: urgência isolada é PROCESSADO, com alerta e um destino só.

Você seguiu o brief, não a spec antiga. É exatamente a decisão que registramos como R1 — e foi a que mais gente errou.

Três coisas que você faz e nós não fazemos

Anotado para não se perder na convergência — vamos trazer para o nosso lado:

+ idade = 0 é tratada como presente, não como ausente (o nosso verificador ainda não testa isso).
+ Conflitos idênticos são deduplicados antes de somar a penalidade.
+ O limiar rejeita NaN e infinito (allow_inf_nan=False).
+ O marcador etapa: "DECISAO_CALCULADA" — o grafo declara que não confirma persistência, que é a leitura mais limpa da RF-30 que vimos até agora.

2. Ajuste 1 — os nomes dos enums

É a mudança mais barata e a de maior efeito: enquanto ela não acontecer, nenhuma resposta da sua API casa com os 79 casos aprovados.

POR QUE ISSO TRAVA A INTEGRAÇÃO

Os 79 fixtures do catálogo trazem o resultado esperado de cada caso já no formato do contrato. Como os valores dos enums são diferentes, a comparação falha em 100% dos casos — mesmo com a lógica correta.

Você mesma está com duas grafias: o README usa FILA_EMERGENCIA_MEDICA e o código usa Fila_Emergencia_Medica. Vale aproveitar para uniformizar nos dois lugares.

Prioridade e destinos

Hoje no seu código	Valor canônico	Referência
"Rotina"	ROTINA	§5.2
"Urgente"	URGENTE	§5.2
"Ambiguidade/Revisão Humana"	NAO_IDENTIFICADA	§5.2, A4
"Fila_Emergencia_Medica"	FILA_URGENCIA_MEDICA	§5.4
"Auditoria_Autorizacoes"	AUDITORIA_AUTORIZACOES	§5.4
"Farmacia_Hospitalar"	FARMACIA_HOSPITALAR	§5.4
"Prontuario_Eletronico"	PRONTUARIO_ELETRONICO	§5.4
"Fila_Revisao_Humana"	FILA_REVISAO_HUMANA	§5.4

SOBRE "AMBIGUIDADE / REVISÃO HUMANA"

Ambiguidade não é prioridade — é cenário (§8.2). A prioridade tem três valores e o terceiro é NAO_IDENTIFICADA, que é justamente o gatilho da decisão A4.

O comportamento que você implementou está certo (prioridade indeterminada entra como campo incerto e gera revisão). Só o nome precisa mudar.

Status

Hoje	Valor canônico	Referência
status: str, default "processado"	enum com 6 valores	§5.3
"pendente_auditoria"	EM_REVISAO	§5.3
"erro"	ERRO_PROCESSAMENTO ou ERRO_PERSISTENCIA	§5.3, §12

Os seis valores: RECEBIDO, PROCESSANDO, PROCESSADO, EM_REVISAO, ERRO_PROCESSAMENTO, ERRO_PERSISTENCIA. RECEBIDO e PROCESSANDO são obrigatórios no fluxo assíncrono (202 + polling, RF-01).

Tipo de documento

O tipo continua sendo str livre na entrada — e pode continuar. A sua função normalizar_tipo() aceita as variações de rótulo, o que é ótimo. A sugestão é só serializar o código estável na resposta: RECEITUARIO_MEDICO, LAUDO_EXAME_LABORATORIAL, LAUDO_EXAME_IMAGEM, GUIA_PROCEDIMENTO, RELATORIO_ALTA_INTERNACAO, ATESTADO_MEDICO, OUTRO, NAO_IDENTIFICADO.

3. Ajustes 2 e 3 — campos e blocos da resposta

Nada aqui muda cálculo: em dois dos três casos o dado já é calculado no seu modelo interno e só não chega à resposta da API.

Campos com formato diferente

Hoje	Formato do contrato	Referência
medicamentos: List[str]	medicamentos: [{nome, dosagem?, frequencia?}]	§9.2, RF-08
exame_realizado: str	exames: List[str]	§9.2
urgencia fora de dados_extraidos	dados_extraidos.urgencia: SIM | NAO | INDETERMINADO	A5
especialidade: str	especialidade: List[str]	§9.2
status_backup default "pendente"	default "sucesso", caminho_objeto opcional	RF-30

Blocos que a resposta não expõe

Bloco	Onde já existe	O que muda
destinos (lista)	DecisaoWorkflow interno	A API devolve só destino_principal. A §8.2 pede os dois destinos, sem reduzir a decisão a um.
qualidade	QualidadeWorkflow interno	Scores, campos ausentes, incertos, conflitos e limiar não saem na resposta — sem isso não dá para auditar nem provar RF-17/RF-18.
erro {codigo, mensagem}	inexistente	Necessário quando o status for de erro (§12).

ALERTA: DE BOOLEANO PARA EVENTO REGISTRADO

Hoje o grafo devolve alerta_simulado: true. A RF-07 e a §8.2 pedem um evento de alerta registrado, não um booleano — e você já tem o modelo Notificacao {canal, mensagem} pronto no schema. É só o grafo preenchê-lo em vez de devolver True.

4. O que ainda falta implementar

Cinco itens. Os dois primeiros são os de maior peso — e o segundo é exatamente a sua parte dos buckets.

1 · ALTO RISCO SEM TEXTO DE URGÊNCIA VAI PARA A FILA URGENTE (RN-03)

Hoje a urgência só é detectada por urgencia = "SIM" ou prioridade URGENTE. A RN-03 exige que achado crítico ou medicamento de alto risco encaminhe para a fila urgente e gere alerta, mesmo sem nenhuma palavra de urgência no texto.

É a única regra de roteamento obrigatória que não existe nem no código nem nos testes. O nosso caso equivalente é Conflito_AltoRisco: receita de varfarina 10 mg, sem texto de urgência, que espera FILA_URGENCIA_MEDICA e status PROCESSADO.

2 · CAMINHO DO BUCKET POR ESTADO (RF-29, RF-30, §6 PASSO 7)

Seu README lista /recebidos, /processados e /auditoria_humana. Falta o subcaminho de urgentes, que entrou no §13 quando aplicamos o bloco de consistência.

Sugestão de derivação a partir da decisão: rotina → processados/ … urgência → processados/urgentes/ … revisão → auditoria_humana/.

A regra do §6 é explícita: documento roteado a revisão tem de ser armazenado no estado/caminho de auditoria. E a RF-30 proíbe URL pública ou credencial na resposta.

Os outros três

- ERRO_PROCESSAMENTO e ERRO_PERSISTENCIA: hoje só existe o marcador de etapa. Falta produzir o status de erro e o envelope {codigo, mensagem} da §12.
- Ingestão (RF-03 + A6): src/ingestion/ está vazio. Nada valida os limites de 10 MB, 20 páginas, extensões e MIME. O limite é inclusivo: 10 MB e 20 páginas passam; 10,5 MB e 21 não.
- documento_id: o grafo exigir está correto internamente — a API é que precisa gerar quando ausente (RN-07). Vale registrar esse limite no README para ninguém se perder.

DOIS PONTOS DE RISCO NA APRESENTAÇÃO

O README promete reduzir a triagem "de 15 minutos para menos de 5 segundos". As metas numéricas ficaram fora do MVP: o critério de aceite são os casos aprovados, com latência medida por etapa e sem meta (decisão A7). Prometer 5 segundos expõe o time a uma cobrança que o critério não define.

O README cita FILA_EMERGENCIA_MEDICA; o nome da §5.4 é FILA_URGENCIA_MEDICA.

5. Seus três cenários, no nosso contrato

Traduzimos os data/samples para o contrato MediFlow v1.0. Os três validam e são ponto fixo das regras — se o contrato divergir, o script quebra.

Cenário	Documento	Status	Destino
1 · Rotina	Hemograma — Maria F. Lima	PROCESSADO	PRONTUARIO_ELETRONICO
2 · Urgência	TC de tórax — TEP agudo	PROCESSADO	FILA_URGENCIA_MEDICA + alerta
3 · Ambíguo	Receita de morfina ilegível	EM_REVISAO	FILA_REVISAO_HUMANA

Dois achados na tradução

- O score de extração de 0,65 do cenário 3 é ilustrativo. Pela fórmula aprovada (A3) ele vale 0,00: são 4 campos obrigatórios ausentes (−0,8) e 2 incertos (−0,2). O desfecho não muda, mas o número precisa ser calculado, não estimado — vale ajustar antes da banca perguntar.
- Se morfina entrar no dicionário de alto risco (A2), o cenário 3 passa a ter DOIS destinos, com alerta: FILA_URGENCIA_MEDICA + FILA_REVISAO_HUMANA. Deixamos isso marcado como pendência, não resolvido, porque depende da lista clínica do time.

Exemplo: o payload do cenário 2 (urgência)

É o mesmo conteúdo do nosso caso Matriz_LaudoDeImagem_Pdf, que já está no kit de demonstração:


```
{
  "status": "PROCESSADO",
  "documento_id": "DOC-CLIN-2026-8942",
  "classificacao": {"tipo_documento": "LAUDO_EXAME_IMAGEM",
                    "especialidade": ["Radiologia", "Pneumologia"],
                    "nivel_prioridade": "URGENTE", "score_classificacao": 0.98},
  "dados_extraidos": {"paciente": {"nome": "Carlos Eduardo Mendes", "idade": 52},
                      "medico": {"nome": "Dra. Renata Silveira",
                                 "crm": {"numero": "145892", "uf": "SP"}},
                      "diagnostico": "Tromboembolismo pulmonar agudo",
                      "cid": "I26.9", "urgencia": "SIM"},
  "qualidade": {"score_extracao": 0.98, "score_combinado": 0.98,
                "limiar_aplicado": 0.85, "campos_ausentes": [], "conflitos": []},
  "decisao_roteamento": {"destinos": ["FILA_URGENCIA_MEDICA"],
                         "destino_principal": "FILA_URGENCIA_MEDICA",
                         "requer_auditoria_humana": false,
                         "notificacao": {"gerada": true, "status": "REGISTRADA"}}
}
```

6. Checklist de alinhamento

Ordem sugerida de execução. As três primeiras destravam o resto e são as mais baratas.

Contrato (mecânico, sem mudar regra)

- [ ] Trocar os valores de NivelPrioridade e DestinoRoteamento pelos canônicos (§5.2, §5.4).
- [ ] Renomear AMBIGUO para NAO_IDENTIFICADA mantendo o comportamento atual.
- [ ] Expor destinos (lista) em DecisaoRoteamento, com destino_principal = destinos[0].
- [ ] Criar o enum StatusTriagem com os 6 valores e o bloco erro {codigo, mensagem}.
- [ ] Expor o bloco qualidade na resposta da API.
- [ ] Trocar alerta_simulado pelo objeto Notificacao que já existe no seu schema.
- [ ] medicamentos como objetos, exames como lista, urgencia dentro de dados_extraidos.

Regras

- [ ] Alto risco sem texto de urgência → FILA_URGENCIA_MEDICA + alerta (RN-03).
- [ ] Derivar o caminho do bucket por estado, incluindo processados/urgentes/ (RF-29, §6 passo 7).
- [ ] Produzir status e envelope de erro nas duas situações de falha (§12).
- [ ] Validar entrada na ingestão: 10 MB, 20 páginas, extensões e MIME (RF-03, A6).

Apresentação

- [ ] Tirar a meta de 5 segundos do README (decisão A7).
- [ ] Uniformizar FILA_URGENCIA_MEDICA no README inteiro.
- [ ] Registrar no README que a API gera o documento_id quando ausente (RN-07).

Como validar do seu lado

```
# no nosso repositório, depois de clonar
python3 tools/validar_cenarios_g10.py     # 3 cenários válidos e estáveis
python3 tests/testar_contrato.py          # 79/79 no contrato
python3 tools/verificar_consistencia.py   # 0 erro; avisos só de arquivos grandes não versionados
```

O QUE LEVAR PARA O SEU REPOSITÓRIO

app/models.py — o contrato executável. Ele recusa payload incoerente e já tem A1, A3, A4, A5 e R1 implementados. Você não precisa reescrever o grafo: pode usar o nosso modelo só como validador da resposta, se preferir.

Os 79 fixtures — cada um com o resultado esperado já no formato do contrato. Servem de corpus de teste de integração.

MAPEAMENTO_DE_CONTRATO.md e ISSUES_PARA_O_G10.md — as tabelas deste PDF em markdown, com as 10 issues prontas para abrir no seu repositório.

Links: o pacote completo (documentação, os 79 fixtures, os PDFs de origem e os geradores) e os dois PDFs atualizados com as decisões A1–A9 estão no dist/ do branch arena/b33f82cc-hackatoon-oracle do repositório berger33/hackatoon_oracle.

Obrigado pelo nível do código — a parte difícil (as regras) você resolveu. O que falta é encaixe de nomes e exposição de dados que você já calcula.